1. Introduction

An **associative array** is a special type of array in SystemVerilog in which elements are accessed using **keys** instead of fixed, sequential indices. Unlike static and dynamic arrays, associative arrays **do not require continuous index values** and their size grows automatically as elements are added.

They are widely used in **verification environments** where data needs to be stored and accessed using **meaningful identifiers** such as names, labels, or categories.

2. Basic Concept

- Each element consists of a **key–value pair**
- The **key** acts as an index
- The **value** stores the data
- Memory is allocated **only when an element is created**

Example idea:

Student name → marks
Module name → usage count
Signal name → toggle count

3. Syntax:

```
data_type array_name [index_type];
int marks[string]; /// string → key type, int → value type
```

4. Types of Index (Key):

(a) Integer key

(b) String key (most commonly used)

(c) Enum key:
```
enum {READ, WRITE, EXEC} mode_t;
int access_count[mode_t];
```

5. Declaration and Initialization:

a. Individual Initialization:
```
score["Ravi"] = 85;
score["Amit"] = 90;
```

b. Aggregate Initialization:
```
score = '{"Ravi":85, "Amit":90, "Neha":88};
```

Note: If a key is accessed before initialization, its value is **automatically 0**.

7. Traversing Associative Arrays

Associative arrays **cannot be accessed using numeric loops** (for i=0).

They are traversed using **foreach**.

```
foreach (score[name])
  $display("%s : %0d", name, score[name]);
```

8. Built-in Methods:

Method	Description
exists(key)	Checks if a key is present
num()	Returns number of entries
delete(key)	Deletes a specific entry
delete()	Deletes all entries
first(key)	Returns first key
last(key)	Returns last key
next(key)	Returns next key
prev(key)	Returns previous key

```
/*A log file generated by an embedded system contains the message "sensor data updated sensor status checked data processed".
Write a SystemVerilog program to analyze this message and maintain an associative array in which each unique log keyword is stored as a key and the corresponding number of occurrences is stored as its value. After processing the complete message, display the keyword-occurrence summary.*/

module word_frequency_log;

  string text = "sensor data updated sensor status checked data processed";
  string word = "";
  int freq[string];

  initial begin
    foreach (text[i]) begin
      if (text[i] != " ")
        word = {word, text[i]};
      else begin
        freq[word]++;
        word = "";
      end
    end
    if (word != "") freq[word]++;

    $display("Word : Frequency");
    foreach (freq[w])
      $display("%s : %0d", w, freq[w]);
  end
endmodule
```

```
26
27 /*  A resource manager maintains usage statistics for different software modules. Write
   a SystemVerilog program that uses an associative array indexed by strings to store the
   execution count of at least eight modules. The program shall print all stored entries,
   determine the module with the highest execution count, and verify whether a given
   module name (for example, "Scheduler") is present in the array.  */
28
29
30 module module_usage;
31
32   int usage[string];
33   string max_mod;
34   int max_val = -1;
35
36   initial begin
37     usage = '{
38       "Parser":12, "Scheduler":25, "Logger":9, "Driver":18,
39       "Monitor":30, "Decoder":14, "Encoder":22, "Reporter":16
40     };
41
42     foreach (usage[k]) begin
43       $display("%s : %0d", k, usage[k]);
44       if (usage[k] > max_val) begin
45         max_val = usage[k];
46         max_mod = k;
47       end
48     end
49
50     if (usage.exists("Scheduler"))
51       $display("Scheduler exists");
52
53     $display("Max usage -> %s : %0d", max_mod, max_val);
54   end
55
56 endmodule
57
```

```
57
58
59  /*A bus  controller operates in different modes such as IDLE, READ, WRITE, and ERROR.
60  Write a SystemVerilog program that uses an associative array indexed by an enum to
    store the number of cycles spent in each mode and displays the stored information. */
61
62  module enum_assoc_array;
63
64    // Define enum type
65    typedef enum {IDLE, READ, WRITE, ERROR} bus_mode_t;
66
67    // Associative array with enum as key
68    int cycle_count[bus_mode_t];
69
70    initial begin
71      // Update cycle counts
72      cycle_count[IDLE]  = 10;
73      cycle_count[READ]  = 25;
74      cycle_count[WRITE] = 18;
75      cycle_count[ERROR] = 2;
76
77      // Display mode-wise cycle count
78      foreach (cycle_count[mode])
79        $display("Mode = %s, Cycles = %0d",
80                 mode.name(), cycle_count[mode]);
81    end
82
83  endmodule
84
```

```
85
86  /* A communication controller operates in multiple states.During simulation, we want to:
87
88  Track how many times each state is entered, Identify the first and last states visited,
89  Iterate through states in order, Check whether a specific state ever occurred Clear statistics after analysis  */
90
91  module fsm_state_analysis;
92
93    // Enum representing FSM states
94    typedef enum {RESET, IDLE, READ, WRITE, ERROR} state_t;
95
96    // Associative array: state -> visit count
97    int state_count[state_t];
98
99    state_t first_s, last_s, curr;
100
101   initial begin
102     // Simulated state transitions
103     state_count[RESET]++;
104     state_count[IDLE]++;
105     state_count[READ]++;
106     state_count[READ]++;
107     state_count[WRITE]++;
108     state_count[IDLE]++;
109     state_count[ERROR]++;
110
111     // Total number of states visited
112     $display("Total unique states = %0d", state_count.num());
113
114     // Check existence of a state
115     if (state_count.exists(READ))
116       $display("READ state occurred");
117
118     // Identify first and last state (enum order)
119     state_count.first(first_s);
120     state_count.last(last_s);
121
122     $display("First state = %s", first_s.name());
123     $display("Last state  = %s", last_s.name());
124
125     // Traverse states using next()
126     curr = first_s;
127     do begin
128       $display("State %-5s -> Count = %0d",
129                curr.name(), state_count[curr]);
130     end while (state_count.next(curr));
131
132     // Clear all collected data
133     state_count.delete();
134     $display("After delete, entries = %0d", state_count.num());
135   end
136
```